

✓ 1. What is Business Logic?

Business Logic is the **core rules** of your application — the logic that defines *how your system should behave* based on business requirements.

✓ It answers the question:

“What should happen?”

🧠 Examples of Business Logic

- Validating that a user is at least 18 years old before creating an account
- Calculating total order price with discounts
- Approving or rejecting a loan
- Checking if a product is in stock before confirming purchase
- Applying tax rules based on location

📌 Where does Business Logic live?

- **Service Layer**
- **Application Layer** (in Clean Architecture)
- **Domain Layer** (Entities, Domain Services)

Business logic does NOT know about SQL, tables, connections, etc.

✗ What Business Logic should NOT do

- Write SQL queries
 - Open DB connections
 - Insert/Update/Delete records
 - Know table names
 - Know where data is stored
-

✓ 2. What is Database Logic?

Database Logic is the logic responsible for **storing, retrieving, updating, deleting** data from the database.

✓ It answers the question:

“HOW should it be stored?”

📌 **Database Logic includes:**

- SQL queries (INSERT, SELECT, UPDATE, DELETE)
- Stored procedures
- Indexing, Joins
- ORMs (EF Core) performing CRUD
- Repositories handling data access

📍 **Where does Database Logic live?**

- **Repository Layer**
 - **Data Access Layer**
 - **Infrastructure Layer** (Clean Architecture)
-

✗ **What Database Logic should NOT do:**

- Perform calculations like discount or eligibility
 - Contain business rules
 - Perform validation like age, email uniqueness, etc.
-

★ **Key Difference Table**

Feature	Business Logic	Database Logic
Purpose	Implements business rules	Manages data storage & retrieval
Focus	What the application must do	How data is handled
Layer	Application/Domain	Repository/Infrastructure

Feature	Business Logic	Database Logic
Tech	C#, Services, Domain Models	SQL, EF Core, Stored Procedures
Knows DB?	✗ No	✓ Yes
Example	“Apply 10% discount to total.” “SELECT price FROM products”	

🔥 Real Example (ASP.NET Core)

User Story:

“When creating a new product, price must not be negative.”

✗ Wrong (Business logic mixed with DB logic)

```
public void AddProduct(Product product)
{
    if(product.Price < 0)
        throw new Exception("Invalid price");

    connection.Open();

    var query = "INSERT INTO Products ..."; // DB Logic inside business logic
}
```

✗ This breaks **Separation of Concerns**.

✓ Correct — Clean Separation

Business Logic (Service Layer)

```
public void CreateProduct(Product product)
{
    if (product.Price < 0)
```

```
throw new Exception("Price cannot be negative");
```

```
_productRepository.Add(product);  
}
```

Database Logic (Repository Layer)

```
public void Add(Product product)  
{  
    _context.Products.Add(product);  
    _context.SaveChanges();  
}
```

- ✓ Business Logic = Validation & rules
 - ✓ DB Logic = Save data
 - ✓ Clean & maintainable
 - ✓ Follows Clean Architecture
-

Why Keeping Them Separate Is Important?

1 Maintainability

Change DB = no impact on business logic.

2 Testability

You can unit test business rules **without database**.

3 Reusability

Same business logic can run with SQL Server, PostgreSQL, or even in-memory DB.

4 Scalability

Clear layers = easier to scale into microservices.

5 Follows Clean Architecture & SOLID principles

Especially **Single Responsibility Principle (SRP)**.

Simple Analogy

Imagine:

- **Business logic = Chef** → Decides recipe, taste, ingredients
- **Database logic = Kitchen staff** → Cuts vegetables, stores ingredients

The chef should NOT cut onions.

The kitchen staff should NOT decide recipes.

If you want, I can also explain:

- ✓ Business Logic Layer vs Application Layer vs Domain Layer
- ✓ Example ASP.NET Core CRUD splitting BL and DB logic
- ✓ How to implement in Clean Architecture
- ✓ Real project folder structure